

Second Assignment: Dataflow analysis

Carmine De Rosa
Manuel Gherardi
Arild Kuti

1 Very Busy Expressions

An expression is defined as a *very busy expression* at a point p if any path starting from p uses the expression without redefining any of its operands. To identify them, we proceed backward, starting from the *Exit* basic block of the Control Flow Graph (CFG), applying the transfer function and using the intersection ($\cap$) as the meet operator.

Parameter	Definition
Domain	Sets of expressions
Direction	Backward: $in[b] = f_b(out[b])$, $out[b] = \wedge in[succ(b)]$
Transfer Function	$in[b] = Gen_b \cup (out[b] \setminus Kill_b)$
Meet Operation ($\wedge$)	Intersection ($\cap$)
Boundary Condition	$in[Exit] = \emptyset$
Initial Interior Points	$in[B] = S$ (Universal set)

Table 1: Very Busy Expressions parameters

Algorithm 1 Very Busy Expressions Algorithm

```

1:  $in[Exit] \leftarrow \emptyset$ 
2: for each basic block  $B \neq Exit$  do
3:    $in[B] \leftarrow S$ 
4: end for

5:  $changes \leftarrow true$ 
6: while  $changes$  do
7:    $changes \leftarrow false$ 
8:   for each basic block  $B \neq Exit$  do
9:      $out[B] \leftarrow \bigcap_{a \in succ(B)} in[a]$ 
10:     $in_{new} \leftarrow Gen_B \cup (out[B] \setminus Kill_B)$ 
11:    if  $in[B] \neq in_{new}$  then
12:       $in[B] \leftarrow in_{new}$ 
13:       $changes \leftarrow true$ 
14:    end if
15:   end for
16: end while

```

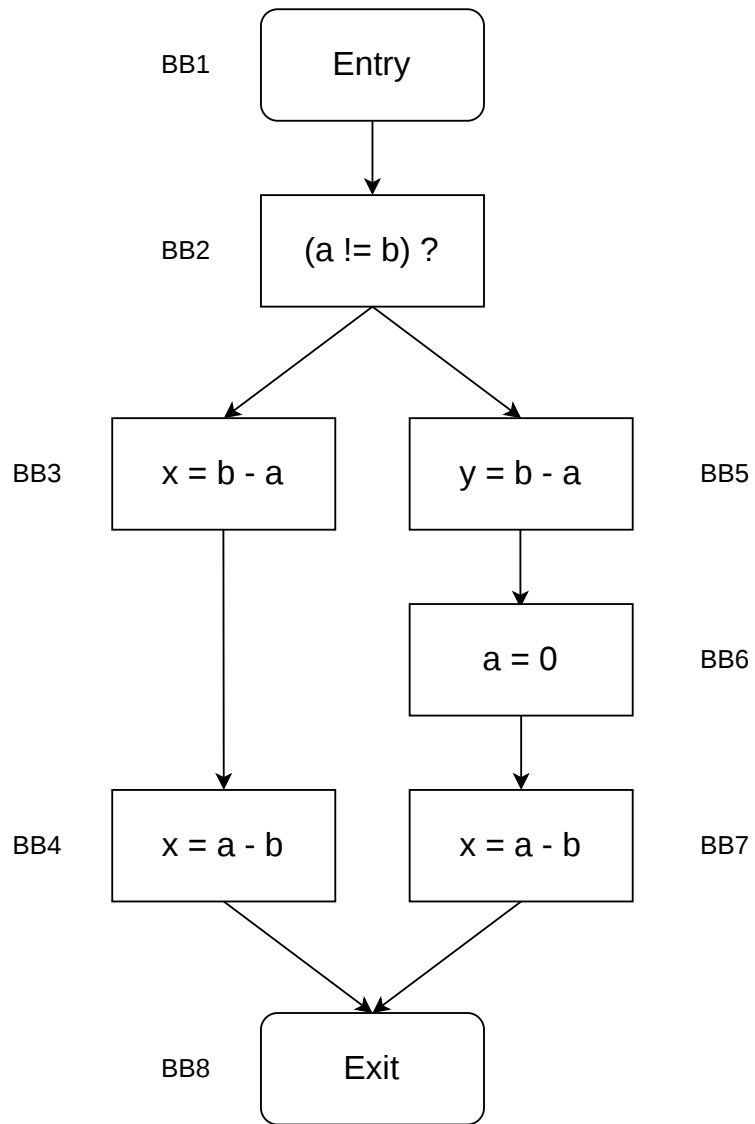


Figure 1: Very busy expression example CFG

Basic Block	Iteration 1		Iteration 2	
	IN[B]	OUT[B]	IN[B]	OUT[B]
BB1 (Entry)		$\{b - a, a! = b\}$		$\{b - a, a! = b\}$
BB2	$\{b - a, a! = b\}$	$\{b - a\}$	$\{b - a, a! = b\}$	$\{b - a\}$
BB3	$\{b - a, a - b\}$	$\{a - b\}$	$\{b - a, a - b\}$	$\{a - b\}$
BB4	$\{a - b\}$	$\emptyset$	$\{a - b\}$	$\emptyset$
BB5	$\{b - a\}$	$\emptyset$	$\{b - a\}$	$\emptyset$
BB6	$\emptyset$	$\{a - b\}$	$\emptyset$	$\{a - b\}$
BB7	$\{a - b\}$	$\emptyset$	$\{a - b\}$	$\emptyset$
BB8 (Exit)	$\emptyset$		$\emptyset$	

Table 2: Iteration table for example CFG (we iterate on BB in decreasing order)

2 Dominator Analysis

L'analisi dei Dominatori è fondamentale per la costruzione della Static Single Assignment (SSA) form e per l'individuazione dei loop naturali nel Control Flow Graph. Un nodo d domina un nodo n se ogni percorso dal nodo di ingresso al nodo n deve passare per d . Spiegazione: Forward perché i dominatori di un nodo dipendono da chi lo precede, non da chi lo segue Transfer function: un blocco aggiunge sé stesso all'insieme dei dominatori in ingresso Boundary: l'Entry domina solo sé stesso all'inizio Meet = intersezione: un nodo d domina b solo se domina tutti i predecessori di b Initial = AllNodes: iniziamo pessimisticamente (ogni nodo è dominato da tutti), poi l'intersezione raffina

Parameter	Dominator Analysis
Domain	Sets of nodes (subsets of all basic blocks)
Direction	Forward: $out[b] = f_b(in[b])$, $in[b] = \bigcap_{p \in pred(b)} out[p]$
Transfer Function	$out[b] = Gen_b \cup (in[b] \setminus Kill_b)$
Meet Operation ($\wedge$)	Intersection ($\cap$)
Boundary Condition	$out[Entry] = \{Entry\}$
Initial Interior Points	$out[B] = AllNodes$ (Universal set)

Table 3: Dominator Analysis as a Dataflow problem

Prima aggiungi subito B (Gen del nodo stesso) Intersechi tutti i predecessori (Meet) Ripeti su tutti i BB finché nessun $Dom(B)$ cambia (convergenza)

Si parte dal massimo possibile (AllNodes) L'intersezione può solo ridurre gli insiemi, mai aumentarli Gli insiemi sono finiti $\rightarrow$ si converge in al più $|AllNodes|$ iterazioni

Algorithm 2 Dominator Analysis

```
1:  $Dom(\text{Entry}) \leftarrow \{\text{Entry}\}$  ▷ Boundary condition
2: for each basic block  $B \neq \text{Entry}$  do
3:    $Dom(B) \leftarrow \text{AllNodes}$  ▷ Pessimistic initialization
4: end for
5:  $changed \leftarrow \text{true}$ 
6: while  $changed$  do
7:    $changed \leftarrow \text{false}$ 
8:   for each basic block  $B \neq \text{Entry}$  do
9:      $new\_dom \leftarrow \text{AllNodes}$ 
10:    for each  $P \in pred(B)$  do
11:       $new\_dom \leftarrow new\_dom \cap Dom(P)$  ▷ Meet: intersection of
12:    end for predecessors
13:     $new\_dom \leftarrow \{B\} \cup new\_dom$  ▷ Transfer function:
14:     $Gen_B = \{B\}, Kill_B = \emptyset$ 
15:    if  $new\_dom \neq Dom(B)$  then
16:       $Dom(B) \leftarrow new\_dom$ 
17:       $changed \leftarrow \text{true}$ 
18:    end if
19:  end for
20: end while
```

3 Constant Propagation

La Constant Propagation è un'analisi dataflow progettata per determinare quali variabili all'interno di un programma hanno un valore costante in un determinato punto dell'esecuzione, permettendo al compilatore di sostituire la variabile con la costante stessa.

Parameter	Definition
Domain	Sets of expressions
Direction	Backward: $in[b] = f_b(out[b])$, $out[b] = \wedge in[succ(b)]$
Transfer Function	$in[b] = Gen_b \cup (out[b] \setminus Kill_b)$
Meet Operation ($\wedge$)	Intersection ($\cap$)
Boundary Condition	$in[Exit] = \emptyset$
Initial Interior Points	$in[B] = S$ (Universal set)

Table 4: Constant Propagation parameters

Algorithm 3 Constant Propagation (TBD)	
1:	$\triangleright$ Algoritmo da definire...